



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto Administrador de BD en consola o
terminal**

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Jahuira Pilco, Dayan Elvis

(2022075749)

Mamani Cori, Cristhian Carlos

(2023077282)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	26/04/2026	Versión Original

Sistema {Nombre del Sistema}
Documento de Visión

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	26/04/2026	Versión Original

INDICE GENERAL

1. Introducción	4
1.1. Propósito	4
1.2. Alcance	4
1.3. Definiciones, Siglas y Abreviaturas	5
1.4. Referencias	5
1.5. Visión General	6
2. Posicionamiento	7
2.1. Oportunidad de negocio	7
2.2. Definición del problema	8
3. Descripción de los interesados y usuarios	8
3.1. Resumen de los interesados	8
3.2. Resumen de los usuarios	9
3.3. Entorno de usuario	9
3.4. Perfiles de los interesados	9
3.5. Perfiles de los Usuarios	10
3.6. Necesidades de los interesados y usuarios	10
4. Vista General del Producto	10
4.1. Perspectiva del producto	10
4.2. Resumen de capacidades	11
4.3. Suposiciones y dependencias	11
4.4. Costos y precios	11
4.5. Licenciamiento e instalación	12
5. Características del producto	12
6. Restricciones	13
7. Rangos de calidad	13
8. Precedencia y Prioridad	13
9. Otros requerimientos del producto	14
10. CONCLUSIONES	16
11. RECOMENDACIONES	16
12. BIBLIOGRAFIA	17
13. WEBGRAFIA	17

Informe de Visión

1. Introducción

1.1. Propósito

El presente documento tiene como propósito definir la visión general del sistema “Administrador de Base de Datos en Consola”, estableciendo sus objetivos, alcance, características principales y los actores involucrados. Sirve como guía para el desarrollo del proyecto y como base para la toma de decisiones durante su implementación.

1.2. Alcance

El sistema permitirá administrar bases de datos relacionales y no relacionales mediante una interfaz de línea de comandos (CLI), ejecutando operaciones como creación de tablas, manipulación de datos (operaciones CRUD) y consultas SQL personalizadas. El sistema se integrará con seis gestores de bases de datos ampliamente utilizados: PostgreSQL, MySQL y SQLite (relacionales), y MongoDB, Redis y Cassandra (NoSQL). Adicionalmente, se distribuirá mediante una extensión de Visual Studio Code, un servidor MCP (Model Context Protocol) y un bot de Telegram en modo de solo lectura.

Dentro del alcance:

- Conexión a bases de datos existentes mediante parámetros de configuración.
- Ejecución de comandos SQL directos (SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER).

- Visualización de resultados en formato tabular legible dentro de la consola.
- Listado de tablas disponibles en la base de datos conectada.
- Manejo básico de errores de sintaxis y conexión.
- Comando de ayuda integrado (help) para consultar instrucciones disponibles.

Comando para salir del programa (exit).

- Conexión y consultas de solo lectura sobre motores NoSQL: MongoDB (find), Redis (comandos de lectura) y Cassandra (CQL).
- Generación de consultas SQL a partir de lenguaje natural mediante IA (con respaldo por patrones si no hay clave de IA configurada).
- Distribución del sistema mediante extensión de VS Code (Marketplace), servidor MCP (PyPI) y bot de Telegram (solo lectura).

Fuera del alcance:

- Desarrollo de un motor de base de datos propio.
- Interfaz gráfica de usuario (GUI).
- Administración de usuarios y permisos a nivel de SGBD.
- Generación de reportes o exportación a otros formatos (CSV, Excel, PDF).
- Conexión simultánea a múltiples bases de datos.

- Funcionalidades de respaldo (backup) o restauración automatizada.
- Operaciones de escritura (INSERT, UPDATE, DELETE, DROP, etc.) desde las integraciones externas: el servidor MCP y el bot de Telegram solo permiten consultas de lectura por motivos de seguridad.

1.3. Definiciones, Siglas y Abreviaturas

Término	Definición
BD	Base de datos
CLI	Command Line Interface
CRUD	Create, Read, Update, Delete
DBMS	Sistema gestor de base de datos
SQL	Structured Query Language
DDL	Data Definition Language
DML	Data Manipulation Language

1.4. Referencias

- Documentación oficial de Python 3.8+
- Documentación oficial de PostgreSQL 13+
- Documentación oficial de MySQL 8.0+
- Documentación oficial de SQLite 3
- Informe de Factibilidad del proyecto (FD01) – Versión 1.0

1.5. Visión General

La visión del proyecto es desarrollar una herramienta de administración de bases de datos basada en consola que permita a los

usuarios interactuar de manera directa, eficiente y controlada con un sistema gestor de bases de datos, sin depender de interfaces gráficas que abstraen los procesos internos.

El sistema busca ofrecer una experiencia simplificada pero funcional, enfocada en la ejecución de comandos SQL directos y estructurados, permitiendo realizar operaciones fundamentales de definición y manipulación de datos. A través de esta interacción, se pretende reforzar la comprensión de los procesos internos de gestión de datos y el uso práctico del lenguaje SQL.

Asimismo, la solución se orienta a un contexto educativo y técnico, donde la prioridad no es competir con herramientas profesionales existentes (como DBeaver, Navicat o pgAdmin), sino proporcionar un entorno controlado que facilite el aprendizaje progresivo de la administración de bases de datos y el desarrollo de aplicaciones basadas en comandos.

La herramienta será compatible con tres sistemas gestores de bases de datos: PostgreSQL, MySQL y SQLite, permitiendo su aplicación en distintos entornos de desarrollo, desde bases de datos ligeras de archivo (SQLite) hasta servidores de producción (PostgreSQL/MySQL).

2. Posicionamiento

2.1. Oportunidad de negocio

Existe una oportunidad significativa en el ámbito educativo y formativo para desarrollar herramientas que permitan a los estudiantes

comprender el funcionamiento interno de las bases de datos sin depender de interfaces gráficas. La mayoría de las herramientas comerciales y de código abierto actuales priorizan la facilidad de uso mediante interfaces visuales, lo que, si bien es útil en entornos productivos, limita la comprensión técnica de los procesos subyacentes.

En el entorno profesional, el uso de interfaces de línea de comandos es predominante en servidores y entornos de automatización, donde la eficiencia, el bajo consumo de recursos y la capacidad de scripting son valorados. Sin embargo, estas herramientas profesionales presentan una curva de aprendizaje elevada.

El proyecto aprovecha esta brecha ofreciendo una solución intermedia: una CLI simplificada, didáctica y funcional, que permite aprender y administrar bases de datos sin las complejidades de herramientas como psql (PostgreSQL) o mysql (cliente MySQL), pero conservando la esencia de la interacción por comandos.

2.2. Definición del problema

Las herramientas actuales de administración de bases de datos suelen abstraer los procesos internos mediante interfaces gráficas, limitando la comprensión técnica. Además, en entornos sin interfaz gráfica, el uso de herramientas de consola puede resultar complejo para usuarios no experimentados.

3. Descripción de los interesados y usuarios

3.1. Resumen de los interesados

Interesado	Rol en el proyecto	Expectativas principales
Docente del curso	Evaluador y supervisor	Que el proyecto cumpla con los requisitos técnicos, documentación completa y funcionalidad demostrable
Estudiantes (equipo de desarrollo)	Desarrolladores	Aprender administración de BD, desarrollar habilidades en Python y CLI, entregar un producto funcional
Estudiantes usuarios	Usuarios finales	Disponer de una herramienta didáctica y fácil de usar para practicar SQL
Universidad	Institución auspiciadora	Proyectos académicos de calidad que aporten valor formativo

3.2. Resumen de los usuarios

Usuario	Descripción	Frecuencia de uso
Usuario básico	Estudiante con conocimientos introductorios de SQL	Sesiones de laboratorio (1-2 veces por semana)
Usuario intermedio	Estudiante o desarrollador que realiza consultas y manipulación de datos	Varias veces por semana
Usuario técnico	Desarrollador o	Uso diario o según

	administrador que utiliza funcionalidades completas del sistema	necesidad
--	---	-----------

3.3. Entorno de usuario

El sistema será utilizado en entornos de desarrollo local, mediante terminal o consola, en sistemas operativos como Windows (CMD, PowerShell, Windows Terminal), Linux (Bash, Zsh) o macOS (Terminal, iTerm2). No se requiere conexión a internet para su funcionamiento básico, salvo para la instalación inicial de dependencias.

3.4. Perfiles de los interesados

Perfil	Responsabilidades
Docente	Define los criterios de evaluación, revisa la documentación, prueba el sistema y asigna la calificación
Estudiante desarrollador	Diseña, implementa, prueba y documenta el sistema según los requisitos establecidos

3.5. Perfiles de los Usuarios

Perfil	Conocimientos requeridos	Interacción típica
Usuario básico	Saber conectarse a una base de datos, escribir comandos SQL simples (SELECT, INSERT)	Escribe comandos con ayuda del comando help
Usuario intermedio	Conocimiento sólido de SQL (joins, subconsultas, DDL)	Ejecuta consultas complejas y administra tablas

Usuario técnico	Conocimiento de administración de bases de datos y de los SGBD soportados	Usa todas las funcionalidades, escribe SQL directo
-----------------	---	--

3.6. Necesidades de los interesados y usuarios

Necesidad	Prioridad
Acceso sencillo a la administración de bases de datos desde consola	Alta
Comprensión del funcionamiento de comandos SQL mediante práctica directa	Alta
Herramienta ligera que no consuma muchos recursos	Media
Interacción rápida mediante consola con feedback claro	Media
Documentación y ayuda integrada para resolver dudas	Alta
Compatibilidad con múltiples motores de base de datos	Media

4. Vista General del Producto

4.1. Perspectiva del producto

El sistema es una aplicación independiente desarrollada en Python 3.8+ que actúa como intermediario entre el usuario y el sistema gestor de base de datos. No reemplaza al SGBD, sino que se conecta a uno existente, interpreta los comandos ingresados por el usuario (que pueden ser SQL directo o comandos internos del sistema) y ejecuta las operaciones correspondientes.

4.2. Resumen de capacidades

- Conexión a bases de datos
- Ejecución de comandos SQL

- Gestión de tablas
- Operaciones CRUD
- Visualización de resultados en consola

4.3. Suposiciones y dependencias

- El usuario cuenta con una base de datos previamente instalada
- Disponibilidad de librerías de conexión en Python
- Acceso a un entorno de ejecución adecuado

4.4. Costos y precios

El proyecto se desarrolla con fines académicos y no tiene un modelo de negocio asociado. Sin embargo, para efectos de simulación profesional, se presentan los costos estimados de desarrollo y los precios de referencia si el producto fuera comercializado.

Costos de desarrollo (tomados del informe de factibilidad FD01):

Categoría	Total (S/.)
Costos Generales	5 040
Costos Operativos	1 060
Costos del Ambiente	320
Costos de Personal	12 000
TOTAL GENERAL	18 420

4.5. Licenciamiento e instalación

El proyecto utiliza Licencia MIT, que permite el uso, copia, modificación, fusión, publicación, distribución, sublicencia y venta del software, siempre que se incluya el aviso de copyright y la licencia en las

distribuciones. Esta licencia es compatible con el uso académico y comercial.

5. Características del producto

MUST (Indispensables)

- Conexión a bases de datos (PostgreSQL, MySQL, SQLite).
- Ejecución de comandos SQL (SELECT, INSERT, UPDATE, DELETE).
- Visualización de resultados en formato tabla.
- Comando de ayuda (help).
- Comando para salir (exit).
- Manejo básico de errores.

SHOULD (Importantes)

- Listado de tablas de la base de datos (tables).
- Información de la conexión activa (info).
- Desconexión de la base de datos (disconnect).
- Limpiar pantalla (cls / clear).
- Conexión y consultas de lectura sobre MongoDB, Redis y Cassandra.

COULD (Opcionales)

- Ayuda específica por comando (help connect).
- Historial de comandos entre sesiones.
- Exportación de resultados a CSV.

- Generación de consultas SQL a partir de lenguaje natural mediante IA (comando preguntar).
- Distribución mediante extensión de VS Code, servidor MCP y bot de Telegram.

6. Restricciones

- El usuario debe tener conocimientos básicos de SQL.
- Se requiere tener instalado previamente el motor de base de datos (PostgreSQL, MySQL o SQLite).
- Las contraseñas se ingresan en texto plano por consola.
- El sistema no cuenta con interfaz gráfica, solo funciona por terminal.
- Dependencia de librerías externas de Python.
- Plazo de desarrollo de 4 meses por ser proyecto académico.
- Las integraciones externas (MCP y Telegram) solo permiten operaciones de lectura, sin excepción.

7. Rangos de calidad

- Usabilidad: Usuario nuevo logra conectarse en menos de 2 minutos usando el comando help.
- Confiabilidad: Tasa de fallos en comandos SQL válidos menor al 1%.
- Rendimiento: Respuesta menor a 1 segundo para consultas simples.
- Portabilidad: Funciona en Windows, Linux y macOS con Python 3.8+.

- **Robustez:** El sistema no colapsa ante errores, permite seguir ingresando comandos.

8. Precedencia y Prioridad

Orden recomendado de desarrollo:

1. Bucle REPL básico (help, exit)
2. Conexión a SQLite (más simple, sin servidor)
3. Ejecución de comandos SQL
4. Visualización de resultados en tabla
5. Conexión a PostgreSQL
6. Conexión a MySQL
7. Comando tables
8. Comando disconnect
9. Comando info
10. Comando cls / clear

9. Otros requerimientos del producto

a) Estandares legales

El desarrollo del proyecto debe cumplir con las siguientes normas:

- **Propiedad intelectual:** El código desarrollado es propiedad de los autores. Se utilizará licencia MIT para permitir uso académico y libre distribución.
- **Software de código abierto:** Todas las librerías utilizadas (Python, mysql-connector, psycopg2, pymongo, redis, cassandra-driver, prompt_toolkit, python-telegram-bot, mcp) son de código

abierto con licencias permisivas, sin necesidad de pago por su uso. La publicación en Visual Studio Code Marketplace, PyPI y la API de Bots de Telegram se realiza conforme a los acuerdos de publicación de cada plataforma, sin costo asociado.

- **Protección de datos:** El sistema no almacena ni procesa datos personales de los usuarios, solo actúa como interfaz para consultar bases de datos externas

b) Estandares de comunicación

- Los mensajes del sistema utilizan iconos claros: ☒ para éxito, **✖** para error, **i** para información.
- El comando help muestra todos los comandos disponibles de forma organizada.
- Los resultados de consultas se muestran en formato de tabla con bordes ASCII.
- Los errores se muestran con mensajes descriptivos y en español, sin mostrar código técnico interno.

c) Estandares de cumplimiento de la plataforma

- **Multiplataforma:** El sistema funciona en Windows, Linux y macOS sin modificaciones.
- **Terminal:** Compatible con cualquier terminal estándar (CMD, PowerShell, Bash, Zsh).
- **Python:** Requiere Python 3.8 o superior instalado en el sistema.
- **Dependencias:** Se incluye archivo requirements.txt para instalar todas las librerías necesarias.

d) Estandares de calidad y seguridad

Calidad:

- Código modular y documentado con docstrings.
- Seguimiento de las convenciones PEP 8 de Python.
- Manejo de errores para evitar que el programa colapse.
- Interfaz de consola clara y fácil de usar.

Seguridad:

- Las credenciales no se almacenan en archivos ni logs.
- Las conexiones se cierran explícitamente al salir o usar `disconnect`.
- Validación básica de comandos internos.
- No se concatenan parámetros del usuario en consultas internas (como la de listar tablas).
- El servidor MCP y el bot de Telegram validan cada comando contra un patrón de solo lectura antes de ejecutarlo, rechazando cualquier operación de escritura.

10. CONCLUSIONES

El proyecto Administrador de BD en consola o terminal es una solución viable que permite a los usuarios interactuar con bases de datos mediante comandos SQL desde una interfaz de línea de comandos.

El sistema cumple con los objetivos planteados: permite conectarse a PostgreSQL, MySQL y SQLite, ejecutar operaciones CRUD, mostrar resultados en formato tabla y manejar errores básicos.

El análisis de factibilidad confirma que el proyecto es viable desde las perspectivas técnica, económica, operativa, legal, social y ambiental.

La herramienta tiene un alto valor educativo, ya que permite a los estudiantes comprender el funcionamiento interno de las bases de datos sin depender de interfaces gráficas.

El código desarrollado es modular y extensible, lo que facilita su mantenimiento y la incorporación de nuevas funcionalidades en el futuro.

11. RECOMENDACIONES

Realizar pruebas con cada uno de los tres motores de base de datos soportados antes de la presentación.

Preparar un script de demostración que muestre una conexión exitosa, una consulta SELECT con varias filas y un ejemplo de manejo de errores.

Verificar que la terminal utilizada durante la presentación soporte caracteres Unicode para que las tablas se muestren correctamente.

Incluir instrucciones claras de instalación y uso en un archivo README.

Para versiones futuras, implementar enmascaramiento de contraseñas usando el módulo getpass de Python.

12. BIBLIOGRAFIA

Ramakrishnan, R., & Gehrke, J. (2003). Sistemas de Gestión de Bases de Datos (3ª ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7ª ed.). McGraw-Hill Education.

Beaulieu, A. (2020). Learning SQL (3ª ed.). O'Reilly Media.

Python Software Foundation. (2026). The Python Standard Library.

13. WEBGRAFIA

Python.org. (2026). *Python 3.8+ Documentation*. Recuperado de

<https://docs.python.org/3/>

PostgreSQL Global Development Group. (2026). PostgreSQL Documentation.

Recuperado de <https://www.postgresql.org/docs/>

Oracle Corporation. (2026). MySQL Documentation. Recuperado de

<https://dev.mysql.com/doc/>

SQLite Consortium. (2026). SQLite Documentation. Recuperado de

<https://www.sqlite.org/docs.html>